

HCI Metrics And Charts Reference

Data: 2026-06-24

Purpose

Este documento consolida:

- todas as métricas HCI principais
- todos os índices dos blocos `Targets` e `Structure`
- as fórmulas usadas pelo motor HCI
- como cada gráfico é montado
- o contrato do IDD e do radar direcional
- a diferença entre motor de cálculo, consulta de exibição e componente gráfico

Double Verification Rule

Toda a conferência abaixo foi feita em duas fontes sempre que possível:

1. motor ou consulta original em `00\_FONTES`
2. implementação atual React ou Kotlin usada pelo app/admin

Quando houver divergência entre “motor de cálculo” e “consulta de exibição”, isso é explicitado.

Primary Sources

App and engine

- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/java/com/example/sportsperformance/logic/HciEngine.kt`
- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/java/com/example/sportsperformance/logic/TargetIntelligenceEngine.kt`

Power Query / VBA contracts

- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/03\_TB\_HCI.txt`
- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/04\_TB\_HCI\_CHARTS\_BUFFER.txt`
- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/05\_HCI\_RHYTHM\_TIMELINE.txt`
- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/06\_HCI\_RHYTHM\_PATH.txt`
- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/07\_HCI\_PERF\_TARGETS.txt`
- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/08\_HCI\_PERF\_STRUCTURE.txt`
- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/11\_TB\_HCI\_OUTPUT\_DECISION\_ENGINE\_INTEGRATED.txt`
- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/15\_TB\_HCI\_TRAINING\_CONSULTATION.txt`
- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/23\_RELATORIO.txt`

Official supporting notes

- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/SportsPerformance\_assets\_hci/hci\_memory/hci\_sports\_performance\_docs\_oficiais\_pt.md`

- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/SportsPerformance\_assets\_hci/hci\_memory/hci\_analise\_alvos.md`
- `00\_FONTES/999\_SportsPerformance\_backup/app/src/main/assets/SportsPerformance\_assets\_hci/hci\_memory/hci\_analise\_alvos\_prescricoes.md`

Current React implementation

- `02\_REACT\_WEB/hci-react-web/src/services/hciDerivedMetricsEngine.js`
- `02\_REACT\_WEB/hci-react-web/src/services/athleteViewMapper.js`
- `02\_REACT\_WEB/hci-react-web/src/components/RhythmMainChart.jsx`
- `02\_REACT\_WEB/hci-react-web/src/components/RhythmPathChart.jsx`
- `02\_REACT\_WEB/hci-react-web/src/components/RhythmTransferChart.jsx`
- `02\_REACT\_WEB/hci-react-web/src/pages/IndicesPage.jsx`
- `02\_REACT\_WEB/hci-react-web/src/pages/ReportPage.jsx`
- `02\_REACT\_WEB/hci-react-web/src/services/reportGenerationEngine.js`

Architecture

Há 3 camadas diferentes:

1. `TB\_HCI` / `HciEngine`

Função: calcular os scores-base do HCI.

2. consultas de apresentação

`HCI\_PERF\_TARGETS`, `HCI\_PERF\_STRUCTURE`, `HCI\_RHYTHM\_TIMELINE`, `HCI\_RHYTHM\_PATH`, `RELATORIO`.

Função: escolher recorte, converter métricas para exibição e montar tabelas consumidas por gráficos e relatório.

3. componentes gráficos

React ou app/admin.

Função: desenhar radar, linha, barras, timeline e relatório visual.

Canonical Event Validity

Fonte 1:

- `03\_TB\_HCI.txt`

Fonte 2:

- `HciEngine.kt`

Regra:

- o evento é lido em ordem de série
- o processamento para no primeiro `STATUS\_EVENTO = FINAL`
- o recorte válido do evento usa as séries até esse ponto

Impacto:

- todos os scores HCI dependem apenas das séries válidas
- gráficos de ritmo e totais devem respeitar o mesmo corte

HCI Overall

Fonte 1:

- `03\_TB\_HCI.txt`

Fonte 2:

- `HciEngine.kt`
- `hciDerivedMetricsEngine.js`

Weighted formula

`HCI\_OVERALL\_SCORE` é montado com os pesos:

- `OUTCOME \* 0.40`
- `PROCESS \* 0.10`
- `RHYTHM \* 0.10`
- `DEEPENING \* 0.05`
- `CONSISTENCY \* 0.10`
- `TRANSFER \* 0.05`
- `RESILIENCE \* 0.05`
- `PRESSURE \* 0.05`
- `PHYSICAL \* 0.05`
- `EMOTIONAL\_NORMALIZED \* 0.05`

Observação:

- `EMOTIONAL\_NORMALIZED` não entra como score bruto 0..10
- ele entra como `EMOTIONAL / 10`

Overall level

Fonte 1:

- `03\_TB\_HCI.txt`

Regra:

- `>= 10.0`: `ELITE`
- `>= 7.4`: `HIGH PERFORMANCE`
- `< 7.4`: `DEVELOPMENT`

Targets Block

Fonte 1:

- `07\_HCI\_PERF\_TARGETS.txt`

Fonte 2:

- `HciEngine.kt`
- `hciDerivedMetricsEngine.js`

Parameters

Ordem oficial do gráfico `Targets`:

1. `OUTCOME`
2. `PROCESS`
3. `RHYTHM`
4. `DEEPENING`
5. `CONSISTENCY`

OUTCOME

Motor:

- referência:
 - rifle: `632.0`
 - pistola: `578.0`
- com mediana de competições:
 - $\text{(((medCompeticoes / outcomeReference) * 10) - 9) * 10}$
- sem mediana:
 - usa o total do evento atual na mesma fórmula

Nível:

- ≥ 8.5 : `ELITE`
- ≥ 6.0 : `ALTO RENDIMENTO`
- ≥ 3.0 : `INTERMEDIÁRIO`
- abaixo disso: `INICIANTE`

PROCESS

Motor:

- sequências válidas de tiros ≥ 9.0
- estatísticas:
 - `maxLen`
 - `count`, limitado a `2`
- fórmula:
 - $\text{(((maxLen * min(count, 2)) / 60) * 10)}$

Nível:

- mesmo corte padrão de `Targets`

RHYTHM

Há duas leituras relevantes.

Leitura A, motor do app:

- fonte:

- `HciEngine.kt`
- `stdAjustado` = std populacional em torno da mediana`
- score-base:
- $10 - (\text{std} * 2.5)$

Leitura B, consulta de exibição do radar:

- fonte:
- `07\_HCI\_PERF\_TARGETS.txt`
- score visual por faixas:
- $\text{std} \leq 0.42$: `10`
- $0.42 < \text{std} \leq 0.50$: transição até `8.5`
- $0.50 < \text{std} \leq 0.90$: transição até `5`
- > 0.90 : queda adicional

Regra crítica:

- o nível de `RHYTHM` é baseado no `STD`, não no score convertido

Faixas de nível:

- $\text{std} \leq 0.42$: `ELITE`
- $\text{std} \leq 0.50$: `ALTO RENDIMENTO`
- $\text{std} \leq 0.90$: `INTERMEDIÁRIO`
- acima disso: `INICIANTE`

DEEPENING

Motor:

- sequência máxima de tiros ≥ 10.0
- fórmula:
- $(\text{maxLen} / 7.0) * 10$

CONSISTENCY

Motor:

- amplitude entre maior e menor `TOTAL\_SERIE`
- fórmula:
- $10 - (\text{amplitude} * 0.7)$

Structure Block

Fonte 1:

- `08\_HCI\_PERF\_STRUCTURE.txt`

Fonte 2:

- `03\_TB\_HCI.txt`
- `HciEngine.kt`

- `hciDerivedMetricsEngine.js`

Parameters

Ordem oficial do gráfico `Structure`:

1. `TRANSFER`
2. `RESILIENCE`
3. `PRESSURE`
4. `EMOTIONAL`
5. `PHYSICAL`

TRANSFER

Motor:

- se existir `HCI\_TRANSFER\_SCORE\_BASE`, usar direto
- fallback de exibição:
 - $\text{(((RESULTADO_EVENTO / HCI_OUTCOME_REFERENCE) * 10) - 9) * 10}$
- em `HciEngine.kt`, o score operacional usa:
 - $\text{(totalEvento / medSimulados) * 10}$
- sem baseline de simulado:
 - score pode ficar `0`

Nível do gráfico `Structure`:

Rifle:

- ≥ 9.95 : `ELITE`
- ≥ 8.7 : `ALTO RENDIMENTO`
- ≥ 4.9 : `INTERMEDIÁRIO`
- abaixo: `INICIANTE`

Pistola:

- ≥ 9.0 : `ELITE`
- ≥ 7.4 : `ALTO RENDIMENTO`
- ≥ 3.4 : `INTERMEDIÁRIO`
- abaixo: `INICIANTE`

RESILIENCE

Motor:

- conta quantas séries melhoram em relação à anterior
- fórmula:
 - $5 + \text{improvements}$
- faixa limitada para `2..10` na consulta de exibição

PRESSURE

Fonte 1:

- `03\_TB\_HCl.txt`

Fonte 2:

- `HciEngine.kt`

Mecânica:

- carga do evento:
 - $\text{pressureLoadEvento} = \text{stdAjustadoRhythm} + (\text{dropCount} * 0.1)$
- referência:
 - mediana da carga de `simulado/treino`
- score:
 - se evento $\leq$ referência:
 - $7 + \text{abs}(\text{diff}) * 3$
 - se evento $>$ referência:
 - $7 - \text{diff} * 3$
- sem referência:
 - 7.0

EMOTIONAL

Motor:

- usa baseline individual:
 - rifle: referência `medTiro` com corte de alerta em -0.2
 - pistola: referência `medTiro` com corte de alerta em -1.0
- cada tiro vira:
 - `LIMPO`
 - `ALERTA`
 - `ERRO`
- grupos consecutivos negativos recebem penalidade
- score final:
 - $10 - \text{penalidadeTotal}$
- faixa limitada:
 - mínimo 3
 - máximo 10

Nível de exibição:

- ≥ 9 : `ELITE`
- ≥ 7 : `ALTO RENDIMENTO`
- ≥ 5 : `INTERMEDIÁRIO`
- abaixo: `INICIANTE`

PHYSICAL

Motor:

- compara mediana da primeira metade com a segunda metade
- se a segunda metade cair mais de `0.2`, registra degradação
- score:
 - `10 - degradation`

Rhythm Timeline

Fonte 1:

- `05\_HCI\_RHYTHM\_TIMELINE.txt`

Fonte 2:

- `RhythmMainChart.jsx`

Data fields

Por série:

- `MEDIA\_SERIE`
- `STD\_SERIE`
- `BREAK\_COUNT`
- `MAIN\_DROP\_DEPTH`
- `MAIN\_DROP\_MARKER`

How it is built

- filtro por atleta + evento + environment + prova
- usa apenas `SR1..SR6`
- calcula:
 - média da série = `TOTAL\_SERIE / 10`
 - `STD\_SERIE` com desvio populacional
 - `PREV\_MEDIA\_SERIE`
 - `BREAK\_COUNT = 1` se a média cair em relação à série anterior
 - `MAIN\_DROP\_DEPTH = PREV\_MEDIA\_SERIE - MEDIA\_SERIE`, quando positivo

Current React visual

`RhythmMainChart.jsx` desenha:

- barras:
 - verde: `STD`
 - vermelho: `Drop Depth`
 - laranja: `Break Count`
- linha azul:
 - mediana da sessão / ponto central por série
- linha vermelha tracejada:
 - predição logarítmica

- linha roxa tracejada:
- referência fixa

Escalas:

- eixo esquerdo: `0..2.5`
- eixo direito:
- rifle: `9.5..10.9`
- pistola: `8.5..10.9`

Rhythm Path

Fonte 1:

- `06\_HCI\_RHYTHM\_PATH.txt`

Fonte 2:

- `athleteViewMapper.js`
- `RhythmPathChart.jsx`

Internal partitions

Cada série é quebrada em:

- `P1 = T1..T3`
- `P2 = T4..T7`
- `P3 = T8..T10`

Cada parte gera:

- `PART\_LABEL`
- `PART\_MEDIA`
- herda contexto da série:
- `MEDIA\_SERIE`
- `STD\_SERIE`
- `BREAK\_COUNT`
- `MAIN\_DROP\_DEPTH`

Current React visual

`RhythmPathChart.jsx` mostra comparação por série com 3 linhas:

- sessão atual
- mediana treino + simulado
- mediana de competição

Transfer Comparison Chart

Fonte 1:

- conceito em `TRANSFER`

- relatórios e comparativos históricos

Fonte 2:

- `RhythmTransferChart.jsx`

Montagem atual:

- linha azul:
 - treino + simulado
- linha vermelha:
 - competição
- linha preta tracejada:
 - tendência logarítmica de transferência

Regras visuais:

- `connectNulls = true`
- pontos aparecem nas linhas reais
- linha de tendência não mostra pontos

Targets And Structure Radars

Fonte 1:

- `07\_HCI\_PERF\_TARGETS.txt`
- `08\_HCI\_PERF\_STRUCTURE.txt`

Fonte 2:

- `IndicesPage.jsx`
- `ReportPage.jsx`

Montagem:

- polígono radial preenchido
- 5 eixos por gráfico
- ordem dos eixos preserva a ordem oficial do relatório
- escala visual `0..10`
- o gráfico mostra score, mas o nível vem da regra de cada parâmetro

Target Intelligence And IDD

Fonte 1:

- `hci\_analise\_alvos.md`
- `hci\_analise\_alvos\_prescricoes.md`
- `hci\_sports\_performance\_docs\_oficiais\_pt.md`

Fonte 2:

- `TargetIntelligenceEngine.kt`

- `reportGenerationEngine.js`

Canonical IDD formula

$\text{`IDD_MEDIO} = \text{QTDE_TIROS} * \text{DIST_MEDIA_MM}$

Canonical directional fields

Por setor/quadrante:

- `QTDE\_TIROS`
- `PERCENTUAL\_TIROS`
- `DIST\_MEDIA\_MM`
- `IDD\_MEDIO`

Canonical reading rule

- o maior `IDD\_MEDIO` define o defeito dominante
- setores adjacentes altos indicam zona de tendência
- setores opostos altos indicam instabilidade geral ou variabilidade de execução

Canonical radial chart

Pistola:

- 8 setores de 45 graus
- cada raio representa `%IDD\_MEDIO` do setor

Rifle:

- 4 quadrantes de 90 graus
- ângulos centrais:
 - `Q1 = 45°`
 - `Q2 = 135°`
 - `Q3 = 225°`
 - `Q4 = 315°`

Current React generated directional report

O relatório direcional React atual usa:

- contagem percentual de direções (`hits / total`)
- não calcula ainda o `IDD\_MEDIO` completo com `DIST\_MEDIA\_MM`

Isso significa:

- o radar direcional atual é um radar de incidência
- o contrato canônico do app original para inteligência de alvo é um radar baseado em `%IDD\_MEDIO`

Report Assembly

Fonte 1:

- `23\_RELATORIO.txt`

Fonte 2:

- `ReportPage.jsx`
- `reportGenerationEngine.js`

Blocos esperados pelo contrato:

- contexto selecionado
- `Targets`
- `Structure`
- qualitativo
- decisão
- treino final
- consulta de treino
- `Rhythm Timeline`
- `Rhythm Path`

No React atual, o relatório premium já agrega:

- visão consolidada
- radar direcional
- `Targets`
- `Structure`
- prescrições
- eventos direcionais

Audit Notes From This Review

Confirmed and aligned

- pesos do `overall`
- normalização de `EMOTIONAL` dentro do `overall`
- `PRESSURE` com referência histórica
- régua do `overall level`
- mapeamento de níveis do viewer

Important distinction to preserve

- `RHYTHM` tem diferença entre score operacional do motor e score convertido para radar na consulta `HCI\_PERF\_TARGETS`
- `IDD` canônico depende de distância radial média; não é apenas contagem de hits por direção
- `TRANSFER` pode ter comportamento diferente entre score operacional e fallback de exibição do radial

Recommended Future Rule

Para qualquer migração futura:

1. validar primeiro o motor base

2. validar depois a consulta de exibição

3. validar por último o gráfico

Nunca assumir que score, nível e gráfico vêm da mesma fórmula única.

File Use Recommendation

Este arquivo deve ser tratado como referência técnica operacional para:

- auditoria de cálculo
- migração para ChatGPT
- recreação de motores em React
- revisão de divergências entre app, admin e viewer